



PROJECT PRESENTATION · FLUTTER APP

Prisma Scan

A mobile app that turns your phone camera into a **document scanner** and a full **PDF studio** — scan pages, clean them up, then read, edit, sign and share as PDF. Built with Flutter for iOS & Android.

Flutter · Dart

Riverpod state

10 screens

Native scanning

Holographic UI

iOS & Android

01 What is Prisma Scan? (in plain words)

Think of it as "**CamScanner meets a PDF editor.**" You point your camera at a paper document; the app finds the paper's edges, straightens it, and makes it look clean — like a real scan. Then you can save it as a PDF and do everything a PDF app does.



Scan

Camera with automatic edge-detection and multi-page capture.



Create

Enhance with filters and export a clean, shareable PDF.



Sign

Read, reorganize, watermark and add your signature.

The problem it solves: people constantly need to turn paper (receipts, IDs, contracts, forms) into tidy digital PDFs — and then tweak or sign them — without a desktop scanner. Prisma Scan does it all on the phone.

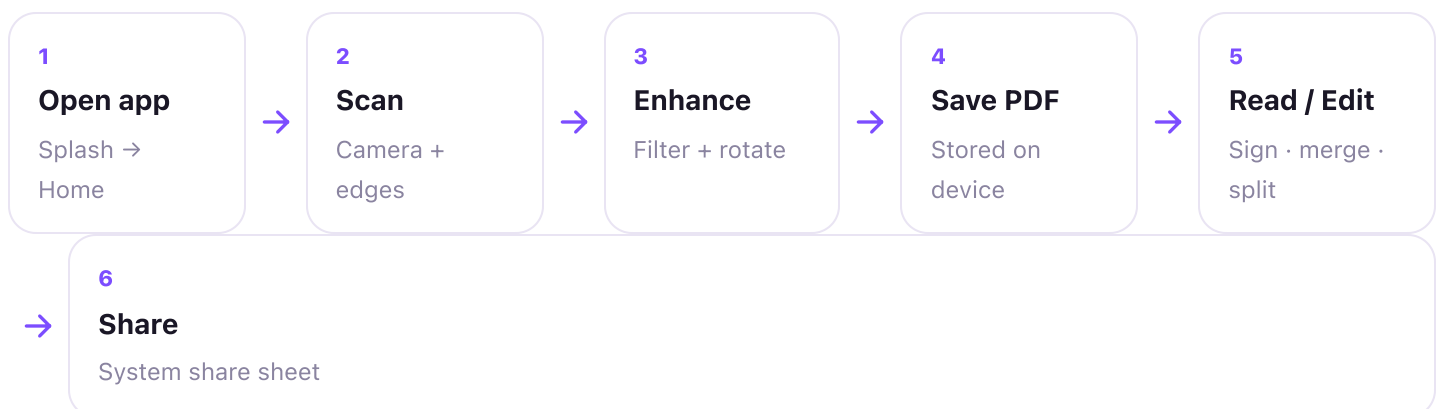
02 The screens

The app has 10 screens. Four are the main tabs; the rest open on top when needed.

#	Screen	What it does
1	Splash	Brand animation while the app loads.
2	Home (tab)	Greeting, search, quick actions (Scan/Import/Tools/Sign), recent documents.
3	Scanner	Live camera with auto edge-detection and batch capture.
4	Edit & Enhance	Crop, filters (Magic/Grayscale/B&W), rotate, reorder → save as PDF.
5	PDF Reader	Read PDFs, pinch-zoom, page thumbnails, bookmark, share.
6	PDF Editor	Reorder/rotate/delete pages, merge, split, watermark, add text, sign, export.
7	Files (tab)	All documents, search, storage meter, rename/share/delete.
8	Tools (tab)	Pick any PDF to open in the editor.
9	Settings (tab)	App lock, cloud sync, export quality, about.
10	Pro Paywall	Subscription plans and premium feature list.

03 How you use it — the main flow

The everyday journey from paper to a shared PDF:



04 How it's built — architecture

The code is organized **feature-first**: each feature (home, scanner, reader...) is its own folder, and inside it the code is split into **4 layers**. This keeps things easy to find and test.

DATA FLOWS TOP → BOTTOM · A TAP ON A SCREEN CALLS A PROVIDER, WHICH CALLS A SERVICE, WHICH RETURNS A MODEL

① Presentation — what you see

Screens (HomeScreen, ReaderScreen...)

Widgets (GlassCard, DocumentTile)

GlassBottomNav + Scan FAB

② Application — the brain (Riverpod)

documentsProvider

settingsProvider

bookmarksProvider

service providers

③ Data — does the heavy work

ScannerService

PdfExporter

PdfEditorService

ImportService

PurchaseService

④ Domain — the plain data

Document

ScanPage

EditorPage

AppSettings

core/ — shared foundation

- theme (colors, gradients, ThemeData)
- router (go_router table)
- utils & constants

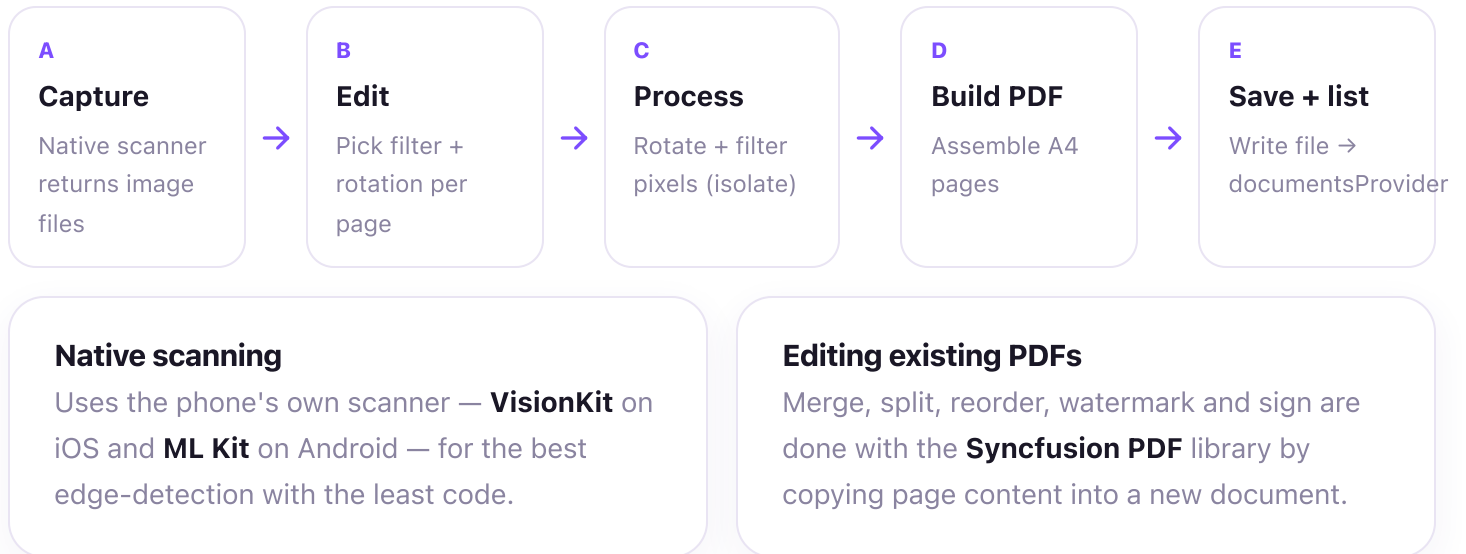
shared/ — reusable widgets

- HoloBackground / SoftBackground
- GlassCard, GradientButton
- GlassBottomNav

Why this design? New features slot in as a new folder without touching others. The UI never talks to native code directly — it always goes through a provider and a service, so any piece can be swapped or tested in isolation (e.g. the mock in-app-purchase service).

05 Under the hood — the Scan → PDF pipeline

This is the app's core magic. The slow image work runs on a **background isolate** (a separate thread) so the screen never freezes.



06 Tech stack & why

Job	Tool	Why
Framework	Flutter	One codebase → iOS + Android, native speed.
State management	flutter_riverpod	Compile-safe, testable, easy to share data across screens.
Navigation	go_router	Central, declarative route table.
Scanning	cunning_document_scanner	Wraps native VisionKit / ML Kit edge-detection.
Make PDF from images	pdf + image	Apply filters & assemble pages in an isolate.
Read / thumbnails	pdfx	Renders PDF pages to the screen & grid.
Edit PDFs	syncfusion_flutter_pdf	Merge/split/reorder/watermark/sign existing PDFs.
Signature	signature	Finger-drawn signature captured as an image.
Import & share	image_picker , share_plus	Bring photos in, send PDFs out via the system.
Storage	shared_preferences , path_provider	Save settings and write PDF files to disk.
Monetization	RevenueCat-ready	Clean interface; mock now, real IAP later.

07 Interview questions & answers

Likely questions about this project, with short answers you can say out loud.

Q Why did you choose Flutter for this app?

Basics

One codebase ships to both iOS and Android with near-native performance, a rich widget set for the custom glassmorphism UI, and easy access to native features (camera, files) through plugins. It cut development time roughly in half versus building two native apps.

Q Why Riverpod instead of setState or Provider?

State

The document list, settings and bookmarks are shared across many screens. Riverpod gives compile-safe, globally-accessible state without `BuildContext`, easy dependency injection (services are providers, so I can swap a real service for a mock in tests), and fine-grained rebuilds with `.select()`. `setState` only fits local, single-widget state.

Q Explain your project's folder structure.

Architecture

Feature-first: each feature folder (home, scanner, reader, editor, files, settings...) contains presentation (UI), application (Riverpod providers), data (services/repositories) and domain (plain models). Shared building blocks live in `core/` (theme, router) and `shared/` (reusable widgets). Features stay independent and testable.

Q The image processing is heavy — how do you keep the UI smooth?

Performance

The pixel work (rotate + filter + build the PDF) runs on a background **isolate** using Flutter's `compute()`. Dart is single-threaded per isolate, so doing this off the UI isolate keeps the main thread at 60fps. A loading overlay shows progress while it runs.

Q How do you scan documents — did you build edge-detection yourself?

Native

No — I use the platform's own document scanner (Apple VisionKit on iOS, Google ML Kit on Android) via a plugin. It already does high-quality edge-detection and perspective correction. Reusing it means better results, less code, and it stays maintained by the OS vendors.

Q How do you edit an existing PDF (merge, split, reorder)?

PDF

With the Syncfusion PDF library. I load the source PDF, then build a new document by importing each page as a template into the order I want; deleted pages are simply skipped, rotation is applied per page, and merge just pulls pages from a second file. Watermark and signature are drawn onto a page's graphics before saving.

Q The whole app is a light "holographic" theme. How did you handle dark-mode devices?

Theming

The design is a deliberate single visual world, so I force the light theme app-wide (`ThemeMode.light`). This avoids the classic bug where a forced-light bottom sheet inherits dark-theme text and becomes invisible — a real issue I caught and fixed by locking the theme.

Q How is the app monetized, and is the payment code done?

Business

Freemium with a Pro paywall (weekly/yearly/lifetime). Payments sit behind a `PurchaseService` interface with a mock implementation that unlocks locally so the flow is fully testable. Going live is just implementing that interface with RevenueCat — no UI changes needed.

Q How do you store documents and settings?

Storage

Generated PDFs are written to the app's documents directory via `path_provider`; each `Document` model keeps the file path. Small key-value settings (app lock, export quality, Pro status) use `shared_preferences`, loaded once at startup and injected through a provider override.

Q What was the hardest bug you fixed?

Debugging

The raised center scan button was being clipped: it lived inside the nav bar's `ClipRRect`, so the part poking above got cut off. I restructured the nav into a `Stack` with `clipBehavior: Clip.none` and overlaid the button on top. Also the dark-mode invisible-sheet bug mentioned above.

Q How would you test this app?

Quality

Unit-test the services and models (PDF export, date formatting, document logic) with mocked providers; widget-test key screens; and use integration tests for the scan→save→read flow. Because services are injected via Riverpod, swapping real ones for fakes is trivial.

Q If you had more time, what would you improve first?

Vision

OCR (searchable text + export to Word), real cloud sync, and folders. Then password-protected PDFs and PDF compression — see the roadmap below.

08 Features we can add next

Grouped by effort so you can plan a roadmap.



Quick wins

low effort

- Empty-state illustrations (already generated)
- Folders / collections in Files
- PDF compression (smaller files)
- Reorder pages via drag in Reader
- Recent-search history
- Haptics & polish animations



Core upgrades

medium effort

- **OCR** — searchable text, copy & export to Word/TXT
- Password-protect / encrypt PDFs
- Biometric app lock (Face ID / fingerprint)
- Free-placement text & annotations (highlight, notes)
- ID-card mode (front + back on one page)
- QR / barcode scanning



Cloud & sync

bigger effort

- Real cloud backup (iCloud / Google Drive / Dropbox)
- Cross-device sync of documents
- Accounts & login
- Share links with expiry
- Team / shared folders



AI & smart features

bigger effort

- AI summary of a scanned document
- Auto-name files from their content
- Smart categories (receipt, ID, invoice)
- Translate scanned text
- Ask questions about a PDF (chat)
- Auto-erase shadows & fingers from scans

Suggested first three: OCR (huge value, expected in scanner apps) → Folders (organization) → Cloud backup (retention & trust). These also strengthen the Pro upsell.